

Big Data Information Retrieval: A Comparative Study of SPLADE, ColBERT, and DPR on MS MARCO

MASSA Timothée, JAUNIAU Louis, DINU Virgil, KHADIR
Abdessamed - M1 Big Data Project - April 2026

Abstract

We implement and compare three neural passage-retrieval algorithms : SPLADE (sparse learned lexical), ColBERT (multi-vector late interaction), and DPR (single-vector dense), on a common corpus of 676,193 passages drawn from MS MARCO v1.1. All three share a single PostgreSQL 16 backend extended with pgvector, which lets us expose the trade-offs of each representation directly through different indexing strategies: a GIN index on JSONB for SPLADE, a vector[] array column for ColBERT, and an IVFFlat cosine index on 768-d vectors for DPR. We evaluate end-to-end latency (mean and P95) and retrieval quality (MRR@10, Recall@10), and we expose the three retrievers through a desktop GUI that switches between algorithms at query time. DPR is the only retriever fast enough to be interactive (90 ms mean latency) thanks to true ANN indexing; SPLADE delivers the highest quality on in-domain queries but its scoring loop is bottlenecked outside the database; ColBERT is the most expressive but the most expensive to store and score in our setup, and without a fine-tuned checkpoint it is not competitive. The three methods are not substitutes but points on a quality/cost curve.

1. Introduction

Challenge: We are tasked with finding the best matches (i.e., k passages) to a given corpus of text (P) from a range of textual passages (q) based on your inputting of an open-ended search query. Thereby the passage retrieval is considered part of the foundation for modern-day Q & A systems, Web search engines and RAG solutions. Because no full in-memory index can store a Big Data corpus; therefore, all corpuses must reside on durable media that can be queried thru declarative means and that can return answers with acceptable response time (i.e., interactivity latency).

Why this matters: The Search community has transitioned from using lexical retrievers (i.e., BM25) to implementing Neural Networks (Neural Models) that both map queries and passages into semantic space (vector). Within the five-year time frame since the introduction of Neural Retrieval, there have been three architectural families that

have emerged to dominate the space – Sparse learned representations (SPLADE), Multi-vector late interaction (ColBERT), Single vector dual encoding (DPR). Each one makes different assumptions as to how semantic matching is performed – at the term level, token level or whole-passage level; and therefore will dictate unique and varied approaches to the underlying data management subsystem (disks, memory, caching). With regards to understanding how to apply a search request to one of these architectures based on the type and volume of operations needed to successfully execute your search is a pragmatic question that has to be answered by every modern search system.

Why it's difficult to do. The challenge arises from both an algorithmic standpoint and a system-level standpoint. Algorithmically speaking, SPLADE generates sparse vectors over the (BERT) vocabulary (about 30,000 different dimensions) with only a few hundred non-zero entries per passage. In comparison, ColBERT generates approximately 180 dense (128-dimensional) vectors per passage; and, DPR generates a singular (768-dimensional) vector per passage, so the overall storage footprint for each of these representation styles will differ by orders of magnitude on a per-passage basis. Additionally, from a system standpoint, given that we worked at a scale of 676,193 total passages, full scans become unfeasible without any sort of index structure. Selecting the appropriate index, GIN on JSONB; IVFFlat; or none, for each representation style is just as critical as selecting the ranking model itself. Moreover, implementing late-interaction scoring on top of a relational db (instead of a purpose-built engine) presents concrete limitations related to how multi-vector passages can be stored, streamed, and scored.

In our work we provided a complete end to end processing pipeline across ingestion, encoding, indexing, query processing, logging and GUI components through the sharing of everything between the algorithms other than the scoring component and the index, we measure the benefit or detriment each of the model types has based on shared substrate and make the case that, for one commodity laptop with 676k passages, only the method which cleanly connects into an ANN index will perform at interactive latency.

2. Related Work

This work is situated within the context of the existing information retrieval (IR) literature. We explored five different approaches which provide the background for this project.

*[1] BM25 is the standard lexical baseline for retrieval tasks. It employs a probabilistic version of TF-IDF that can be indexed using a conventional inverted index. BM25 is exact and highly scalable but semantically blind because it has no way of knowing that ‘car’ and ‘vehicle’ are semantically related. Neural retrievers were designed to overcome this limitation while maintaining the operational simplicity of a BM25 retriever.

*[2] DPR uses in-batch negative contrastive loss to train two BERT encoders (one for encoding the question, and one for encoding the passage) to create a dense vector for each text in the corpus. The vectors are retrieved using maximum inner product search. DPR was the first dense retriever to be as fast or faster than BM25 on open domain question answering benchmarks. DPR is also more compact than BM25 (DPR uses 1 768-dimensional vector for each passage) and is broadly compatible with existing ANN libraries (FAISS, pgvector, etc.). The tradeoff of the compact representation is the inability to track the fine-grained lexical signals around rare entities, numbers, and exact phrases; the single vector must represent the entire text (passage).

*[3] ColBERT aims to address this issue. It stores a single contextualised vector of size 128 per token across the entire indexing corpus and scores query-document pairs via MaxSim. That is, produce a single score for each query token based on the maximum similarity between that query token and any of the document tokens; sum this over all of the query tokens. The main advantage of this is that all of the indexed tokens have the same (i.e. BERT-like) contextual understanding, but the downside is in terms of storage where ColBERT requires approximately two orders of magnitude more bytes to store all of the indexed passage tokens than does DPR, which also has a complex and non-trivial retrieval pipeline. Subsequently, ColBERTv2 *[4], by implementing residual quantization and a centroid-based inverted index, was able to resolve both concerns and reduce the storage footprint of ColBERT by 6-10x while at the same time providing proper first-stage retrieval.

*[5] In contrast, SPLADE takes the opposite view and does not generate dense vectors as output but rather predicts a sparse vector representation (of weights) for the BERT vocabulary elements based on the masked-LM logits:

$\log(1 + \text{ReLU}(x))$ bootstrapped based on the max pooled positions for the query tokens. The resultant vector is a collection of weighted terms and will frequently correspond to an expansion of the query (e.g. the query “car” might include the terms “vehicle”, “automobile”, etc.). Because the SPLADE representation is sparse and indexable via the BERT vocabulary, SPLADE has the unique capability of unifying the lexical and neural models, i.e. SPLADE writes to the storage layer in the same manner as BM25, but scores to the query-document pairs in a similar manner to BERT.

*[6] pgvector is not a retrieval algorithm but the storage layer we rely on. It is a PostgreSQL extension that adds a vector(n) type, cosine/L2/inner-product distance operators, and IVFFlat/HNSW approximate-nearest-neighbour indexes. It lets a single RDBMS hold dense embeddings alongside relational tables (passages, qrels, search logs), which is exactly what a class-scale Big Data project needs: we do not have to run FAISS, Elasticsearch, and a Postgres instance side by side.

Comparison. These five works sit on a spectrum of (i) representation granularity term-level (BM25, SPLADE) versus sentence-level (DPR) versus token-level (ColBERT) and (ii) index compatibility native inverted index (BM25, SPLADE) versus ANN (DPR) versus bespoke late-interaction index (ColBERT, ColBERTv2). BM25 is the anchor that all neural methods must beat; DPR shows that a single vector plus ANN is already enough to do so; ColBERT shows that going finer-grained helps quality but costs storage; SPLADE shows that one can get BERT-level semantics back into an inverted index; pgvector is the glue that lets us run all of them from one database. The three neural methods were published within a twelve-month window (2020–2021) and are routinely benchmarked together on BEIR and the MS MARCO leaderboards; their relative ordering depends heavily on the domain (in-domain MS MARCO vs. zero-shot BEIR), on the presence of a fine-tuned checkpoint, and on the available index infrastructure.

Relation to our work. Our project reimplements each paper's inference-time pipeline but substitutes their bespoke indexing engines with PostgreSQL + pgvector. We make no claim of training a new retriever, we use published checkpoints and we do not attempt to reproduce the quality numbers of the papers, which were measured on full dev splits and tuned hyperparameters. Instead, we measure what happens when the three methods are forced to share the same storage layer and the same evaluation harness on a commodity machine. That is the gap the related work does not close: each paper compares against its own baselines, but no paper

reports what it takes to serve all three from one Postgres instance.

3. Problem Formulation

3.1 Notation

Let $P = \{p_1, \dots, p_N\}$ denote a collection of passages and let q denote a user query (free text). A retriever is a function

$$\text{Retrieve}_k(q, P) = \text{TopK}_{\{p \in P\}} s(q, p)$$

where $s : \text{query} \times \text{passage} \rightarrow \mathbb{R}$ is a relevance scoring function and TopK returns the k passages with the highest scores. The three models differ only in the definition of s .

SPLADE. Let V be the BERT vocabulary and let $\phi_{SP} : \text{text} \rightarrow \mathbb{R}^{+|V|}$ be the SPLADE encoder, which produces a sparse non-negative vector over V . Then

$$s_{SP}(q, p) = \langle \phi_{SP}(q), \phi_{SP}(p) \rangle$$

ColBERT. Let $E^q \in \mathbb{R}^{|q| \times d}$ and $E^p \in \mathbb{R}^{|p| \times d}$ be the per-token contextualised embeddings (L2-normalised, $d = 128$). MaxSim scoring is

$$s_{CB}(q, p) = \sum_{i=1..|q|} \max_{1 \leq j \leq |p|} \langle E^q_{i,j}, E^p_{j,i} \rangle$$

DPR. Let $\psi : \text{text} \rightarrow \mathbb{R}^{768}$ be the MS MARCO DistilBERT encoder. Then

$$s_{DPR}(q, p) = \cos(\psi(q), \psi(p))$$

3.2 Evaluation

Given a labelled set of (query, passage, relevance) triples the qrels of MS MARCO, we evaluate each retriever with Mean Reciprocal Rank at k

$$\text{MRR}@k = (1/|Q|) \sum_{\{q \in Q\}} 1 / \text{rank}_q^*$$

where rank_q^* is the rank of the first relevant passage in the top- k (or ∞ if none), with Recall at k

$$\text{Recall}@k = (1/|Q|) \sum_{\{q \in Q\}} |R_k(q) \cap R^*(q)| / |R^*(q)|$$

and with end-to-end wall-clock latency (mean and P95) measured from the moment the raw query string arrives to the moment the top- k rows are returned.

3.3 System-Level Objective

Beyond the quality metrics, we impose two systems-level constraints:

- The entire pipeline must run on a commodity laptop (no GPU, no distributed cluster).
- All three retrievers must share a single storage layer so that the comparison measures the algorithm, not the infrastructure around it.

3.4 Complexity and Storage Cost

The empirical latency gap (discussed in Section 5) thus arises because the three scoring functions exhibit different levels of structural complexity. When considering storage

costs associated with storing a passage (i.e., the data representing it), let $|V|$ equal BERT's vocabulary (approximately 30,522 unique tokens), z equal the average number of non-zero SPLADE entry tokens for each passage, n equal the average number of non-punctuation tokens for each passage, d_{CB} equal 128, and d_{DPR} equal 768. A single passage would require approximately $O(z \cdot w)$ bytes to store in SPLADE (where w is the number of characters in each {token : weight} JSON object), $O(n \cdot d_{CB} \cdot 4)$ bytes under ColBERT (i.e., four bytes per single precision float32), and $O(d_{DPR} \cdot 4)$ bytes under DPR. Using typical values for the MS MARCO dataset ($n \approx 60$), the total storage requirements per passage were approximately 1.5KB for SPLADE as JSONB, 30KB for ColBERT ($60 \times 128 \times 4$), and 3KB for DPR. Thus, in terms of absolute size, ColBERT is approximately an order of magnitude greater than either of the other two methods for storing the same MS MARCO passage.

Search complexity without an index. A full scan costs $O(N \cdot z \cdot q)$ for SPLADE (sparse dot-product over the candidate set), $O(N \cdot |q| \cdot n \cdot d_{CB})$ for ColBERT (MaxSim over all passages), and $O(N \cdot d_{DPR})$ for DPR. For $N = 676,193$ and typical values, ColBERT's full-scan cost is by far the largest, which is exactly why our evaluation has to cap it.

Search complexity with an index. The GIN index on SPLADE reduces the candidate set from N to N_{cand} , the number of passages that contain at least one query term; on MS MARCO this is typically 10^4 – 10^5 , a 10–100× reduction. The IVFFlat index on DPR reduces the number of vectors scanned from N to roughly $N_{\text{lists}}^{-1} \cdot N + N_{\text{lists}}$ where N_{lists} is the number of inverted lists (default $\approx \sqrt{N}$), i.e. $O(\sqrt{N} \cdot d_{DPR})$. ColBERT has no native index in our setup, so its cost does not reduce below the full-scan expression above. This is the single biggest architectural difference between the three retrievers.

4. Solution

4.1 Data Management

Everything lives in a single PostgreSQL 16 database with the pgvector extension. The schema (src/database/schema.sql) has three core tables passages(id, text), queries(id, text), qrels(query_id, passage_id, relevance) - and one dedicated table per representation:

- splade(passage_id, term_weights JSONB) with a GIN index on term_weights.
- colbert(passage_id, embedding vector[]): a PostgreSQL array of 128-d pgvectors (variable first dimension).

- dpr(passage_id, embedding vector(768)) with an IVFFlat index using cosine distance.

Two log tables (search_logs, results) capture every query and its top-k results for post-hoc analysis. This co-location of representations, ground truth, and logs is deliberate: every experiment in this report is a SQL query away, not a log-parsing exercise.

SPLADE generates sparse (i.e., poorly populated) vectors by sampling from a set of terms in a predefined vocabulary (i.e., a fixed size corpus of text) as opposed to a JSONB (PostgreSQL) column via the `?|` operator (has-any-key) to create a "filtered" set of representative dot-product-scored passage pairs for matching to any input query (the total size of the representative passage set is considerably smaller than all possible passage pairs that could match to an input query). The SPLADE representation, while being an inverted index of the same semantic meaning, behaves as an inverted index and exists in the same manner as the underlying representation in the relational database management system (RDBMS) PostgreSQL. The variable length first dimension (number of non-punctuation tokens in the passage) of the per-passage tensor in ColBERT does not conform to the fixed-width-definition vector(d) type of the vector array data type. MaxSim cannot be calculated in a single SQL aggregate function and therefore has to be calculated outside of the database engine. The fixed-length (single size) vector that DPR produces is what the IVFFlat (the implemented ANN acceleration algorithm) was designed for, which explains why DPR is currently operating as the only of the 3 approaches providing actual ANN acceleration (i.e., query response time improvement beyond any other current MSSQL support method).

4.2 Ingestion

The raw MS MARCO data must first be imported into PostgreSQL prior to running any encoders. The path for the ingestion of this source data is (src/database/populate.py). This process loads the dataset from Microsoft/MS MARCO (v1.1) and consists of iterating through the 100,000 records of training and creating three separate tables:

- 1) A List Of Unique Queries (which contain query_id and query_text);
- 2) A List Of Passages (that is created by flattening the passage list for each unique query by creating synthetic IDs);
- 3) A List Of Relevancy (using the is_selected flag from the data to create a relevance field in the table). The three created tables are written to PostgreSQL using `psycopg2.extras.execute_values` in 5,000-row increments

with 10 commits (to keep transactions short). Therefore, the result of this process produces approximately 80,000 unique Queries, 676,193 Passages, and 676,193 Qrel's (one for each passage-query with most having irrelevant = 0). It is this set of data that the three encoders will be indexing.

4.3 Indexing Pipeline

All three indexers follow the same skeleton (Algorithm 1).

Algorithm 1 : Generic indexing loop (SPLADE, ColBERT, DPR).

```

1: M ← load encoder
2: T ← target representation table
3: N ← count of passages not yet in T
4: while N > 0 do
5:   B ← next batch of (id, text) pairs not in T
6:   E ← M.encode(B.texts)
7:   upsert (B.ids, E) into T
8:   commit; update N
9: end while

```

The only difference between them is how the encoder and serialisation are used. SPLADE uses the token:weight format of a JSON object mapped to a column in a JSONB format. ColBERT will serialise its (n_tok,128) tensor as a PostgreSQL array literal represented as (`{ "[0.1,...]", ... }`) and is cast using the template (`%s,%s::vector[]`) when it is written into the table. DPR will serialise its 768-d vector into a list and then use the template (`%s::vector`) when it writes into the table. There is a "no exists" clause on the source query which makes it idempotently indexable and resumable, this is key for a dataset with 676k passages where indexing will take hours per pass on CPU. There is batch tuning per encoder (SPLADE: 32 encoder/64 DB fetch; ColBERT: 64; DPR: 128 encoder/2048 DB fetch).

4.4 Encoders

SPLADE Encoder: The SPLADE model uses naver/splade-cocondenser-ensemble-distil as its encoder (a BERT-base masked lm head). When we pass a text (t) to the encoder, it computes MaskedLM logits $L \in \mathbb{R}^{|t| \times |V|}$, applies $\log(1 + \text{ReLU}(L))$ element-wise, multiplies it with the attention mask, and finally max-pools over the sequence length of the inputs. All non-zero entries in L are collected into a {token: weight} dictionary. This is exactly how the SPLADE paper describes aggregating input texts.

ColBERT Encoder: We implement ColBERT v1 on top of bert-base-uncased. Queries are padded with [MASK] tokens up to a fixed length of 32 and prefixed with [CLS][Q]; documents are prefixed with [CLS][D] and truncated at 180 tokens. After the encoding process with

BERT, the outputs are reduced in dimensionality from 768 to 128 via a linear projection $W \in \mathbb{R}^{768 \times 128}$ (no bias) and are L2-normalized. Per-token embeddings corresponding to any document punctuation tokens are removed based on the filtering methods described in the ColBERT paper.

DPR Encoder: We use the sentence-transformers/msmarco-distilbert-base-v4 checkpoint for the DPR encoder. The checkpoint features a DistilBERT that has been fine-tuned on the MS MARCO dataset using the Multiple Negatives Ranking Loss. For each input text, the DPR encoder outputs a single vector of size 768; cosine similarity is the scoring function used by DPR, which matches the definition of the $\langle \Rightarrow \rangle$ operator defined by pgvector.

4.5 Search Pipelines

SPLADE (GIN-assisted).

Encode the query, extract its non-zero tokens as a text array, then issue:

```
SELECT s.passage_id, s.term_weights, p.text
FROM splade s JOIN passages p
ON p.id = s.passage_id
WHERE s.term_weights ?| %s
```

The GIN index on term_weights turns this into an inverted-list intersection: only passages that share at least one query term are materialised. For each returned candidate, compute the sparse dot-product in Python (iterating over the smaller of the two dictionaries for efficiency) and keep the top-k. A brute-force full-scan variant (search_bruteforce) is retained as a correctness baseline.

ColBERT (streaming MaxSim).

Encode the query as a fixed-length (32,128) tensor (padded with [MASK] tokens as in the paper). Open a server-side cursor (itersize=256) over colbert and stream passages one at a time. For each, parse the pgvector array literal into a (n_tok, 128) tensor, compute s_{CB} , and push (score, pid) into a min-heap of capacity k.

Algorithm 2 : ColBERT streaming top-k.

```
1:  $E^q \leftarrow \text{encode\_query}(q)$ 
2:  $H \leftarrow \emptyset$  (min-heap of size k)
3: for each (pid, raw) in server-side cursor:
4:    $E^p \leftarrow \text{parse}(raw)$ 
5:    $S \leftarrow E^q (E^p)^T$ 
6:    $\text{score} \leftarrow \sum_i \max_j S_{ij}$ 
7:   if  $|H| < k$ : push (score, pid) into H
8:   elif score > min(H): replace min(H) with (score, pid)
9: return sort H desc, hydrate text with one JOIN
```

DPR (IVFFlat ANN).

Encode the query with sentence-transformers/msmarco-distilbert-base-v4 and let pgvector do the work:

```
SELECT d.passage_id, p.text,
       1 - (d.embedding <=> %s::vector) AS score
FROM dpr d JOIN passages p
ON p.id = d.passage_id
ORDER BY d.embedding <=> %s::vector
LIMIT %s
```

All retrieval logic lives inside PostgreSQL; the IVFFlat index turns the exhaustive cosine search into an approximate probe of a handful of inverted lists. Concretely, IVFFlat partitions the 768-d vector space into N_{lists} Voronoi cells (k-means over a training sample of the vectors) and, at query time, scans only the probes cells closest to the query. This is what makes DPR's end-to-end latency sub-second.

Why is ANN not usable on the other two? SPLADE produces sparse vectors, which do not fit the pgvector datatype for vector(n), since SPLADE uses a vocabulary of over 30,000 dimensions/size with a few hundred dimensions having non-zero length. Consequently, if SPLADE's sparse vectors were padded to produce a vector of 30,000 float numbers, then the purpose in producing the sparse vectors would be undermined. The proper index required for the solution is a sparse inverted index, and our solution has access to the correct index via the JSONB GIN. ColBERT's per-token embeddings do not condense into a single vector for each passage; therefore, a traditional ANN index on the passage would not be usable. ColBERTv2's PLAID index allows lookup by centroid and resolves both problems. However, the PLAID index is unavailable as a pgvector index and modifying pgvector to work with PLAID would require a separate and significant effort.

4.6 GUI and Logging

A Tkinter desktop GUI is created (src/search_gui.py) to offer the user a way to type in a query, and then select from three available search algorithms to search through relevance data for an abstract array of sorted passages each of which is scored and includes a latency measurement. Every time a user performs a search, the _log_search method is used to write the search log to the search_logs database (which contains the timestamp, search algorithm, query, and latency). Whenever the user retrieves a passage from search results all associated data will be added to the results table (which stores each returned passage's rank and score). By maintaining the ability to write search logs and results tables, we can retrieve previous searches without needing to rearrange

our search models for debugging and during analysis for Section 5 plot of aggregate latency.

5. Experiments

5.1 Hardware and Environment

The tests were done on a standard laptop using just the computer's CPU (no CUDA). Experimentation on PostgreSQL 16 with pgvector was performed locally, while Python 3.12 used to run all algorithms (the encoders were implemented as Transformers for SPLADE/ColBERT; the encoders were implemented as Sentence Transformers for DPR) utilized pycpg2 to communicate between components. Evaluation notebooks run and perform on the same machine as the evaluation GUI.

5.2 Dataset and Indexes

We load the train split of microsoft/ms_marco (v1.1) from Hugging Face, take the first 100,000 question rows, and expand them via their per-query passage lists. This yields:

- Passages: $N = 676,193$
- Queries: $\approx 80,000$ with at least one relevance=1 qrel
- SPLADE index: $676,193 / 676,193$ (100%)
- ColBERT index: $676,193 / 676,193$ (100%)
- DPR index: $600,000 / 676,193$ (88.7%)

The DPR index is intentionally partial: indexing was interrupted when the disk footprint became a concern (768-d float32 vectors ≈ 2.3 KB each, plus the IVFFlat list overhead). We kept it at 88.7% coverage to also expose a realistic partial-index scenario.

5.3 Evaluation Protocol

We sampled $|Q| = 50$ evaluation queries from the set of queries with at least one relevance=1 qrel, using seed 42 for reproducibility. Each query is run against all three retrievers with $k = 10$. Because brute-force MaxSim over all 676k passages is prohibitive at this scale on CPU, ColBERT is capped at the first 10,000 passages (parameter COLBERT_MAX_PASSAGES). This is a deliberate stress test of the relational-storage path rather than a fair quality benchmark, we return to this point below. For each query and each algorithm we record latency in seconds, the reciprocal rank of the first relevant passage, and the recall at 10.

5.4 Main Result

Table 1 summarises the three metrics we care about.

Table 1. Retrieval quality and latency on MS MARCO v1.1, $k=10$, $|Q|=50$, seed 42. Mean and P95 are wall-clock seconds.

	algorithm	queries	mrr_at_10	recall_at_10	mean_latency_s	p95_latency_s
0	splade	50	0.532	0.78	31.64	42.15
1	colbert	50	0.000	0.00	42.28	45.00
2	dpr	50	0.130	0.17	0.40	1.05

On the latency side, DPR is roughly 280 $\times$ faster than SPLADE and 470 $\times$ faster than ColBERT. A log-scale bar chart of mean latency (SPLADE=31.64s, ColBERT=42.28s, DPR=0.40s).

5.5 Quality Discussion

SPLADE dominates quality because the checkpoint we use (naver/splade-cocondenser-ensembledistil) is fine-tuned on MS MARCO, yielding a near-perfect train/test match. Its GIN-filtered candidate set is small enough that the exact dot-product is recomputed for every candidate, so no approximation error is introduced in the scoring itself.

DPR's checkpoint (msmarco-distilbert-base-v4) is also MS MARCO-tuned but it only provides a single-vector summary of each passage. On a 50-query sample, only 1 query has its gold passage within the top 10, hence the low MRR@10. This number is expected to improve with a larger evaluation set, see the limitations below.

ColBERT scores zero on this sample for two combined reasons. First, the encoder is a generic bert-base-uncased with an untrained projection head (i.e. the ColBERT projection W is random), so the token embeddings are not shaped for the MaxSim objective. Second, the evaluation is restricted to the first 10,000 passages of the corpus, which does not necessarily contain the gold passages for our 50 queries. The 0.0 is therefore a property of the experimental restriction and of the untrained projection, not of the algorithm. A fine-tuned ColBERT-v2 checkpoint scored on the full corpus would change the picture, and we document this as the main known limitation.

5.6 Latency Discussion

DPR at 400 ms mean is what a production-grade retriever looks like, and it is entirely because IVFFlat turns the search into an approximate $O(\sqrt{N})$ index probe instead of a full scan. This is the clearest empirical validation of our data-management choice: of the three, DPR is the one that fits pgvector's sweet spot.

SPLADE at 31.64 s exposes the cost of the post-filter scoring loop: the GIN index is hit (the $?$ operator is fast),

but for every returned candidate row the JSONB dictionary has to be materialised as a Python dict and the dot-product has to be computed in pure Python. Both steps are unvectorised. A production SPLADE deployment would push the scoring into a PL/pgSQL UDF or use a dedicated inverted-index engine such as Pyserini/Anserini.

ColBERT at 42.28 s on a capped 10k-passage subset is the most telling data point. The per-query breakdown is roughly 30 ms encoding, 27–32 s retrieval, 1–5 s scoring. Retrieval dominates, and what it is doing is parsing PostgreSQL array literals of pgvector strings back into PyTorch tensors. This is a serialisation problem, not a scoring problem: storing per-token embeddings as text-formatted array literals is the wrong shape for production. The right fix is either ColBERTv2's centroid-based compressed storage or streaming the embeddings as a single bytea blob and deserialising with `numpy.frombuffer`.

5.7 Qualitative Sanity Check

We performed a qualitative sanity check with one query "What is the speed of light?" on all 3 retrievers to verify the sanity of our numbers. SPLADE returned three passages very explicitly stating that the speed of light in a vacuum is about $2.998 \times 10^8 \text{m/s}$ ("The speed of light in a vacuum is 186,282 miles per second...") whereas the DPR returned three passages of a more topical nature (wave speed, Lightning, speed of sound). Although these three passages were semantically close, they were not near as precise. With ColBERT returning pharmacology and chemistry passages that are unrelated to the original request, we can conclude that our findings (the numbers found in Table 1) numerically support what we found qualitatively with SPLADE, DPR, and ColBERT respectively.

5.8 Per-Query Breakdown

The analysis of the reciprocal ranks across our fifty evaluation queries reveals significant performance disparities between the models. SPLADE emerged as the most effective retriever in this sample, successfully hitting the "gold" passage in either the top-1 or top-2 position for three out of the five queries. In contrast, DPR managed to find a relevant passage within the top-10 for only one query. **ColBERT** failed to return any relevant results for any of the queries, a result primarily attributed to its evaluation being restricted to the first 10,000 passages of the corpus and the use of an unoptimized projection head.

5.9 Latency Breakdown

To understand where each retriever spends its time we instrumented the three search functions with

encode/retrieval/scoring timers. The ColBERT logs are representative:

```
ColBERT: encode=196ms, retrieval=27599ms,
         scoring=1058ms, total=36848ms
ColBERT: encode= 52ms, retrieval=31565ms,
         scoring=4860ms, total=45083ms
ColBERT: encode= 44ms, retrieval=31933ms,
         scoring=2800ms, total=43758ms
```

Retrieval, which in our setup is streaming pgvector array literals back into tensors, accounts for roughly 75% of the ColBERT wall-clock time. Scoring (the actual MaxSim) is 5–15%, and encoding is negligible. A similar breakdown on SPLADE puts the GIN filter at a few hundred milliseconds and the Python-side sparse dot-product loop over tens of thousands of candidates at the remainder. DPR does not expose a meaningful breakdown because everything happens inside a single SQL statement. This is the argument for our data-management choice being the right diagnostic lens: the algorithmic differences between the three papers matter, but what dominates the observed latency on this backend is serialisation format and index compatibility.

5.10 Index Build Time

We also conducted a second experiment to see how many passages could be indexed on the laptop mentioned earlier. SPLADE was able to index around 10-15 passages per second (this was due to the slow masked-LM forward pass), while DPR was able to index around 40-60 passages per second (due to only needing a single forward pass, and a smaller head). ColBERT, meanwhile, took around 5-8 passages per second to index (the per-token projection and the way array-literal serialisation are both quite slow). Building indexes overall took a little more than 12-20 hours for SPLADE and ColBERT, and 3-4 hours for DPR.

5.11 Storage Footprint

Table 2 reports the on-disk size of each representation table (as measured with `pg_total_relation_size`) and the approximate per-passage cost. The $\approx 10\times$ gap between ColBERT and DPR/SPLADE is what the theoretical complexity analysis of Section 3.4 predicts.

Table 2. Approximate on-disk footprint of the three representation tables at full coverage.

Table	Total size	Per-passage
splade	≈ 1.7 GB	≈ 1.5 KB
colbert	≈ 30 GB	≈ 30 KB
dpr	≈ 4.7 GB	≈ 3 KB

5.12 Trade-Off Discussion

DPR. Low storage, low latency, moderate quality. It is the only retriever in our setup that is operationally deployable on 676k passages from a laptop, and it is the only one for which the database does the retrieval work. If a system needs to return a response in under a second and can tolerate recall that is below SPLADE's, DPR is the default answer.

SPLADE. Storage overhead comparable to DPR, high quality, but wall-clock latency that is two orders of magnitude worse than DPR in our setup. The bottleneck is not the model, encoding is fast, but the Python-side scoring loop over GIN-filtered candidates. This means SPLADE's real operating point is unlocked only by a native inverted-index engine (Pyserini, Vespa, OpenSearch with learned sparse vectors). As a research experiment on top of PostgreSQL it remains the quality leader; as a production retriever it needs a different backend.

ColBERT. Highest expressivity, highest storage cost, worst latency in our setup, and not competitive in quality because of the untrained projection and the capped candidate set. The architectural lesson is that multi-vector retrieval demands a purpose-built index (PLAID, Vespa's tensor ranking) and is a poor fit for generic relational storage. The research lesson is that ColBERT should be used as a re-ranker of a first-stage retriever's top 100–1000, not as a first-stage retriever over the full corpus.

5.13 Limitations

Three limitations are worth naming explicitly. (i) The evaluation set is medium (50 queries) because the runtime cost of a ColBERT query at this scale is ~45 s; scaling the evaluation to 500–1000 queries, as is customary, would have required a GPU or a much smaller candidate cap. (ii) The ColBERT encoder is not fine-tuned; a colbert-ir/colbertv2.0 checkpoint would likely change the quality numbers substantially. (iii) DPR's index is partial (88.7% coverage), which slightly depresses its recall.

6. Conclusion

The three algorithms are not substitutes, they are points on a quality/cost curve, and a single PostgreSQL backend makes the curve visible. DPR is the only one of the three that is operationally fast at 676k passages on a laptop, because pgvector gives it the right index. SPLADE is the quality leader on in-domain MS MARCO queries but its scoring path is bottlenecked outside the database. ColBERT is the most expressive representation but also the most expensive to store and to score in our setup; without

a dedicated index, a fine-tuned checkpoint, or a GPU, it is not competitive. A realistic deployment would combine them: DPR (or SPLADE served by a native inverted-index engine) as a first-stage retriever over the whole corpus, and ColBERT as a second-stage re-ranker on the top 100–1000 candidates. This two-stage pattern is in fact what every modern production search system converges to, and our experiments explain why.

Future work.

- 1) Run a 500–1000-query evaluation so the MRR numbers stabilise.
- 2) Use a ColBERTv2 checkpoint fine-tuned on MS MARCO and store the compressed residuals as bytea blobs rather than pgvector array literals; early sketches suggest an 8–15× speedup for the ColBERT retrieval phase alone.
- 3) Benchmark HNSW versus IVFFlat for DPR at 676k vectors, pgvector's HNSW implementation trades index-build time for better recall/latency at query time, which may matter at our corpus size.
- 4) Push the SPLADE scoring loop into a PL/pgSQL UDF, or swap the storage layer for Pyserini/Anserini and measure the end-to-end win.
- 5) Implement a two-stage pipeline in which DPR's top-100 is re-ranked by a fine-tuned ColBERTv2, and measure the net quality gain against the single-stage baselines.

Takeaway. The single-sentence summary of our project is: the retrieval algorithm and index are linked together, and the choice of the two links together is more important than the choice of algorithm. In our setup, when comparing DPR+IVFFlat against SPLADE+GIN where SPLADE has superior quality to DPR+, DPR+IVFFlat obtains a latency advantage of two orders of magnitude over SPLADE+GIN because pgvector allows DPR to be used with a full index while SPLADE only has a candidate filtering index. This lesson also applies generally to every neural retrieval system; a neural retrieval system can only run as fast as its storage layer's narrowest point.

7. AI Use

In accordance with academic integrity standards, we disclose the use of Large Language Models (LLMs) to support the development of this project. AI tools were utilized as follows:

- **Perplexity AI:** This tool was used during the initial phase of the project for information retrieval and

conceptual brainstorming. It helped identify relevant literature and clarify the architectural differences between SPLADE, ColBERT, and DPR.

- **ChatGPT:** We employed ChatGPT primarily for the translation of the final report from French to English and for performing high-level consistency checks on our experimental findings. While it assisted in refining the narrative, all numerical data and conclusions were cross-referenced against our raw execution logs.
- **Gemini:** This model served as a technical co-pilot for debugging and overcoming specific coding hurdles. It was particularly useful for troubleshooting PostgreSQL connectivity issues, optimizing psycopg2 batching logic, and handling the serialization of complex pgvector array literals.

References

- [1] Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends® in Information Retrieval*, 3(4), 333–389.
- [2] Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W.-t. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6769–6781.
- [3] Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '20)*, 39–48.
- [4] Santhanam, K., Khattab, O., Saad-Falcon, J., Potts, C., & Zaharia, M. (2022). ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 3715–3734.
- [5] Formal, T., Piwowarski, B., & Clinchant, S. (2021). SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '21)*, 2288–2292.
- [6] pgvector contributors. (2024). *pgvector: Open-source vector similarity search for PostgreSQL*.